

23.2 -Capstone Project Workshop: End-to-End Application Development

Name:B.Sarayu

Hno:2303A51842

Smart Student Attendance & Performance Tracker:

Task-1 :Project proposal

Objective:

To develop a web-based application that helps colleges track student attendance, monitor performance, and generate reports using AI-assisted tools.

Features

- Student login & admin login
- Mark attendance (daily/subject-wise)
- View attendance percentage
- Alerts for low attendance
- Performance tracking (marks)
- Dashboard with charts
- Export reports (PDF/Excel)

Tools & Technologies:

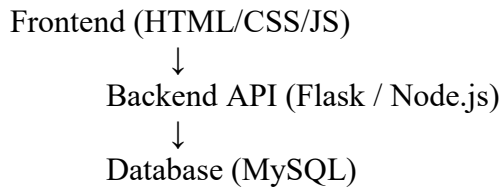
- **Frontend:** HTML, CSS, JavaScript (or React)
- **Backend:** Python (Flask) / Node.js
- **Database:** MySQL / SQLite
- **AI Tools:** ChatGPT, GitHub Copilot
- **Deployment:**
 - GitHub Pages (frontend)
 - Render / Heroku (backend)

Expected Output

- A working web application
- Dashboard showing attendance stats
- Admin can manage students
- Deployed live project link

Task 2 – Application Design

Architecture (Simple)



Database Tables

- Students (id, name, dept)
- Attendance (id, student_id, date, status)
- Marks (id, student_id, subject, score)

UI Pages

- Login Page
- Dashboard
- Attendance Page
- Reports Page

Task 3 – Development Plan

Phase 1 – Frontend

- Create forms (login, attendance)
- Dashboard UI

Phase 2 – Backend

- API:
 - /login
 - /mark-attendance
 - /get-attendance
 - /get-report

Phase 3 – Integration

- Connect frontend with backend using fetch/AJAX

Task 4 – Documentation

Include in README.md:

- Project description

- Setup steps
- Technologies used
- Screenshots

Task 5 – Testing

Use AI to:

- Generate test cases
- Check errors
- Improve performance

Example:

- Test login validation
- Test attendance calculation

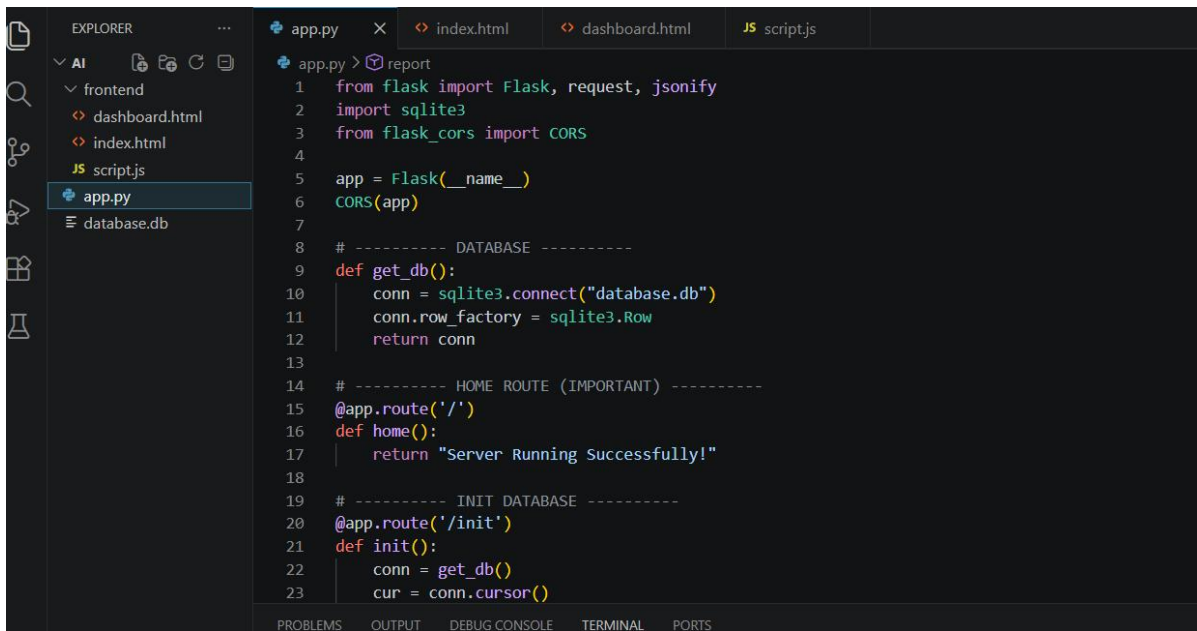
Task 6 – Deployment:

- Upload code to GitHub
- Deploy:
 - Frontend → GitHub Pages
 - Backend → Render / Heroku

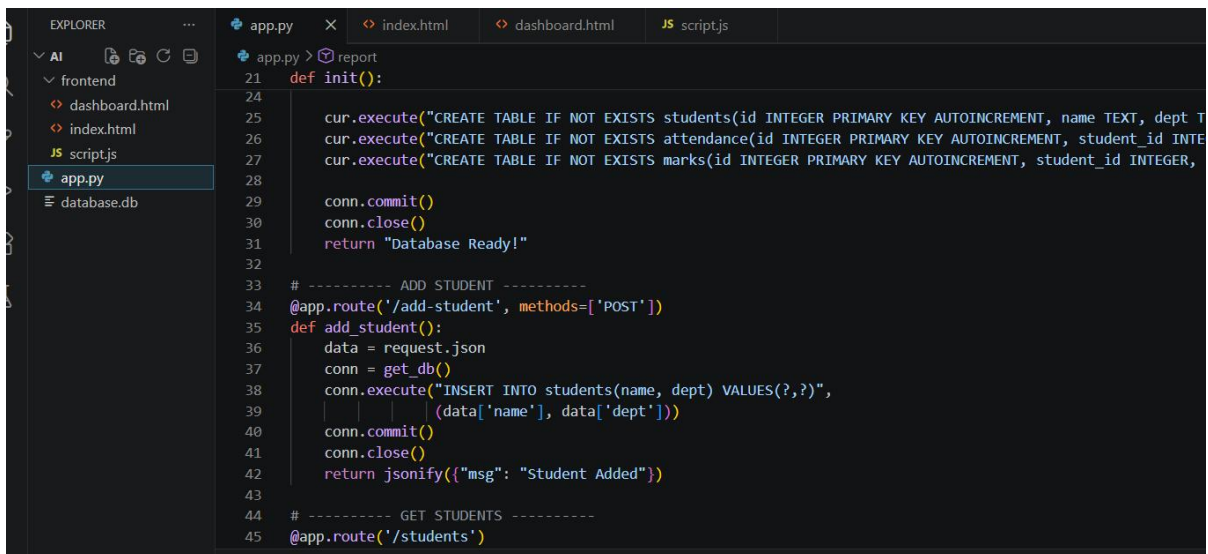
```
attendance-tracker/
├── backend/
│   ├── app.py
│   └── database.db (auto created)
├── frontend/
│   ├── index.html
│   ├── dashboard.html
│   ├── script.js
│   └── style.css
└── README.md
```

Backend (Flask API):

attendance_tracker/backend/app.py



```
app.py > report
1 from flask import Flask, request, jsonify
2 import sqlite3
3 from flask_cors import CORS
4
5 app = Flask(__name__)
6 CORS(app)
7
8 # ----- DATABASE -----
9 def get_db():
10     conn = sqlite3.connect("database.db")
11     conn.row_factory = sqlite3.Row
12     return conn
13
14 # ----- HOME ROUTE (IMPORTANT) -----
15 @app.route('/')
16 def home():
17     return "Server Running Successfully!"
18
19 # ----- INIT DATABASE -----
20 @app.route('/init')
21 def init():
22     conn = get_db()
23     cur = conn.cursor()
```



```
app.py > report
21 def init():
24
25     cur.execute("CREATE TABLE IF NOT EXISTS students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT, dept TEXT)")
26     cur.execute("CREATE TABLE IF NOT EXISTS attendance(id INTEGER PRIMARY KEY AUTOINCREMENT, student_id INTEGER, date TEXT, status TEXT)")
27     cur.execute("CREATE TABLE IF NOT EXISTS marks(id INTEGER PRIMARY KEY AUTOINCREMENT, student_id INTEGER, score INTEGER, date TEXT)")
28
29     conn.commit()
30     conn.close()
31     return "Database Ready!"
32
33 # ----- ADD STUDENT -----
34 @app.route('/add-student', methods=['POST'])
35 def add_student():
36     data = request.json
37     conn = get_db()
38     conn.execute("INSERT INTO students(name, dept) VALUES(?,?)",
39                 (data['name'], data['dept']))
40     conn.commit()
41     conn.close()
42     return jsonify({"msg": "Student Added"})
43
44 # ----- GET STUDENTS -----
45 @app.route('/students')
```

Frontend:

attendance_tracker/frontend/index.html (Add Student Page)

```
app.py index.html X dashboard.html JS script.js
frontend > index.html > html
1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Student App</title>
5 </head>
6 <body>
7
8 <h2>Add Student</h2>
9
10 <input id="name" placeholder="Name">
11 <input id="dept" placeholder="Department">
12 <button onclick="addStudent()">Add</button>
13
14 <br><br>
15 <a href="dashboard.html">Go Dashboard</a>
16
17 <script src="script.js"></script>
18
19 </body>
20 </html>
```

attendance_tracker/frontend/dashboard.html

```
D: > AI Assisted Coding > attendance_tracker > frontend > dashboard.html > html
2 <html>
7 <body>
8 <h2>Attendance Dashboard</h2>
10
11 <button onclick="loadStudents()">Load Students</button>
12
13 <ul id="studentList"></ul>
14
15 <button onclick="submitAttendance()">Submit Attendance</button>
16
17 <h3>Report</h3>
18 <button onclick="getReport()">Get Report</button>
19 <ul id="report"></ul>
20
21 <script src="script.js"></script>
22 </body>
23 </html>
```

attendance_tracker /frontend/script.js

```
app.py  index.html  dashboard.html  JS script.js  X
frontend > JS script.js > ...
1  const API = "http://127.0.0.1:5000";
2
3  function addStudent() {
4      const name = document.getElementById("name").value;
5      const dept = document.getElementById("dept").value;
6
7      fetch(API + "/add-student", {
8          method: "POST",
9          headers: {"Content-Type": "application/json"},
10         body: JSON.stringify({name, dept})
11     })
12     .then(res => res.json())
13     .then(data => alert(data.msg));
14 }
15
16 function loadStudents() {
17     fetch(API + "/students")
18     .then(res => res.json())
19     .then(data => {
20         const list = document.getElementById("list");
21         list.innerHTML = "";
22     });
23 }
```

OUTPUT:

Database Ready!

Add Student

[Go Dashboard](#)



Dashboard

Add Marks

Report

Dashboard

Load Students

Submit Attendance

Add Marks

ai assisted

90

Add Marks

Report

Get Report

127.0.0.1:5500 says

Select student first!

OK